

IMPLEMENTATION & INTEGRATION SPECIFICATION

JWT Security & React Frontend Integration

Technical Analysis of Stateless Token Security Architecture, JJWT Filter Chain, and Modern Glassmorphic SPA Integration

SECURITY PROTOCOL

Stateless JWT (HMAC-SHA256)

FRONTEND FRAMEWORK

React 18 + Vite 5 SPA

DATE & VERSION

August 2026 • Release 2.0

1. Modernization & Architectural Scope

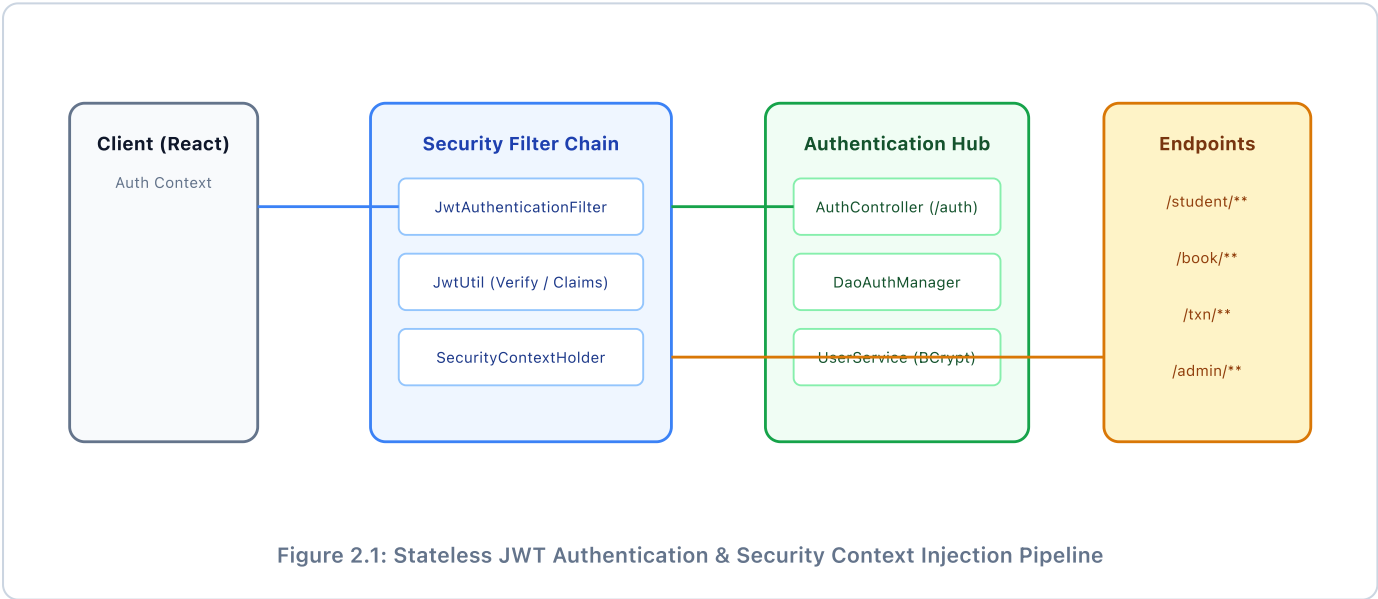
This report details the architectural transformation of the Digital Library Management System from legacy HTTP Basic session-based authentication to a modern, decoupled **Stateless JWT (JSON Web Token) Security Model** and a state-of-the-art **React Single Page Application (SPA)**.

Key Engineering Deliverables

- Stateless Security Pipeline:** Replaced server-side HTTP sessions with HMAC-SHA256 signed JWT tokens (24h TTL) and automated Bearer token request validation.
- Role-Based Access Control (RBAC):** Gated route protection separating Student functions from Admin operations with custom 401/403 handlers and CORS support.
- Interactive React + Vite Frontend:** Built a glassmorphic dashboard in `frontend/` featuring dynamic quota tracking, 1-click book returns with fine calculations, and multi-role portals.

2. JWT Security Architecture & Pipeline

The security subsystem utilizes the **JJWT (Java JSON Web Token) 0.12.6** library suite, configured to operate in a fully stateless mode.



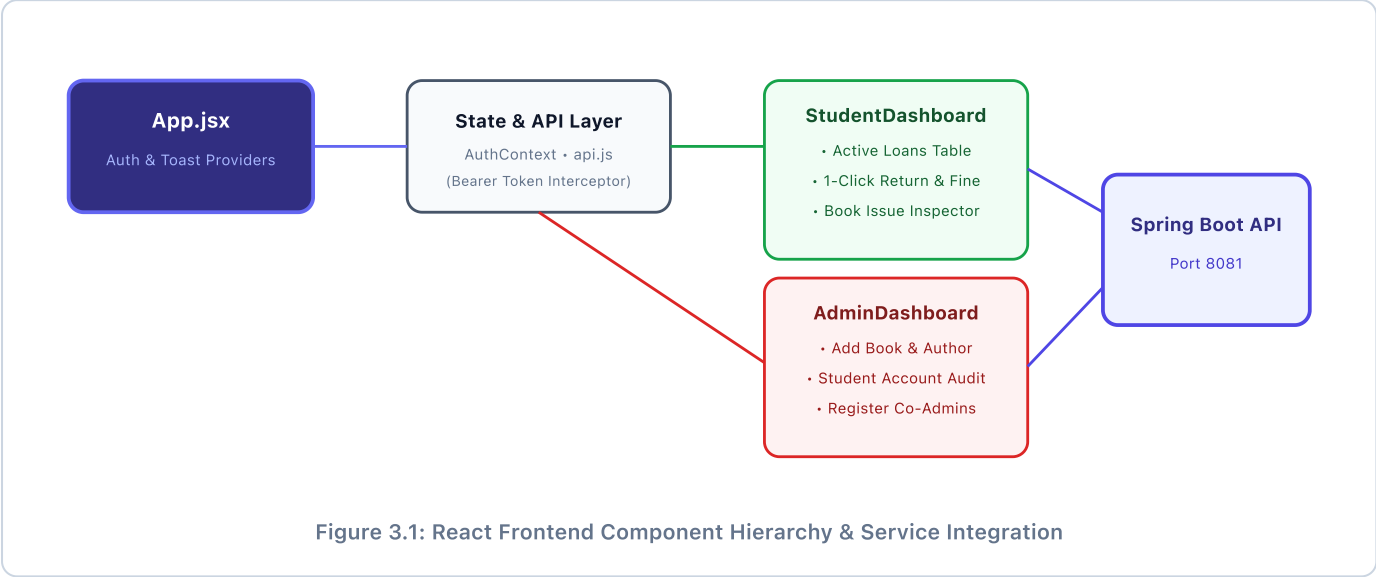
2.1 Component Responsibilities

Component	File Location	Core Responsibility
JwtUtil	<code>configs/JwtUtil.java</code>	Generates HMAC-SHA256 tokens with claims (<code>sub</code> , <code>roles</code>), extracts claims, checks expiration, and validates tokens.
JwtAuthenticationFilter	<code>configs/JwtAuthenticationFilter.java</code>	<code>OncePerRequestFilter</code> intercepting <code>Authorization: Bearer <token></code> , resolving <code>UserDetails</code> and setting <code>SecurityContextHolder</code> .

Component	File Location	Core Responsibility
JwtAuthEntryPoint	<code>configs/JwtAuthenticationEntryPoint.java</code>	Catches unauthenticated requests and returns standard HTTP 401 JSON error format.
GlobalExceptionHandler	<code>configs/GlobalExceptionHandler.java</code>	Catches validation errors, duplicate username/email conflicts, and runtime exceptions.
SecurityConfig	<code>configs/SecurityConfig.java</code>	Stateless session policy, CORS policy, and role route gating (STUDENT vs ADMIN).

3. React Single Page Application (SPA) Architecture

The frontend is developed in `frontend/` using **Vite 5** and **React 18**. It features a custom, responsive design system using **Vanilla CSS** with glassmorphism, responsive data tables, modal dialogs, and real-time toast alerts.



3.1 Key Frontend Features

- **Universal Auth Modal (3 Tabs):** Seamlessly switch between Sign In, Student Sign Up, and Admin Sign Up.
- **Live Quota Tracker:** Visualizes student quota (3 books max), 15-day borrowing window, and ₹100/day overdue fine rate.
- **1-Click Book Return:** Calls `POST /txn/initiate?bookId={id}&transactionType=RETURN`, displays generated UUID transaction code, and updates loan tables in real-time.
- **Admin Control Center:** Multi-field book & author creator with genre dropdowns, student record auditor by ID, and co-admin generator.

4. Verification, Testing & Execution Guide

The complete full-stack integration was verified through automated test suites, direct REST API invocations with cURL, and live browser testing.

4.1 Automated & Functional Verification Summary

Operation	Endpoint / Target	Payload / Params	Status	Result
Student Registration	POST /student/create	{"username": "student_1", ...}	200 OK	User created, Student assigned with ACTIVE status.
Admin Registration	POST /admin/create	{"username": "superadmin", ...}	200 OK	Admin entity created with ROLE_ADMIN authority.
JWT Login	POST /auth/login	{"username": "...", "password": "..."} Authorization: Bearer <token>	200 OK	Signed JWT token returned with 24h expiration.
Profile Fetch	GET /student/get	Authorization: Bearer <token>	200 OK	Returns student profile & active books array.
Add Book (Admin)	POST /book/create	{"name": "Design Patterns", ...}	200 OK	Book & Author created with genre enum.

4.2 How to Run Locally

1. Run Spring Boot Backend (Port 8081):

cd /Users/apple/pulkit/jbdl-95/JBDL95/JBDL95Minor2/JBDL95MINOR1 && ./mvnw spring-boot:run
2. Run React Frontend (Port 3000):

cd /Users/apple/pulkit/jbdl-95/JBDL95/JBDL95Minor2/frontend && npm install && npm run dev